28

Epilogue

Don't let it end like this. Tell them I said something.
—Pancho Villa

You've arrived at the end of the journey. Nice work.

We hope you can find some valuable takeaways from this book. We suggest the list should include the following:

1. What architecture is, why it's important, what influences it, and what it influences.
2. The role that architecture plays in a business, technical, project, and professional context.
3. The critically important symbiosis between architecture and quality attributes, how to specify quality attribute requirements, and the quick immersion you've had into the dozen or so of the most important QAs.
4. How to capture the architecturally significant requirements for an architecture.
5. How to design an architecture, document it, guide an implementation with it, evaluate it to see if it's a good one for your needs, reverse-engineer it, and manage a project around it.
6. How to evaluate an architecture's cost and benefit, what it means to be architecturally competent, and how to use architecture as the basis for an entire software product line.
7. Architectural concepts and patterns for systems on the current technological frontier: edge applications and the cloud.

Fine. Now what?

It's very tempting to end the book with a cheery imperative to go forth and architect great systems. But the truth is, life isn't that simple.

The authors of this book have, collectively, an embarrassingly large number of years of experience in teaching software architecture. We teach it through books like this one, in the classroom to students, and in industrial short courses

to practicing software architects of all stripes, from "aspiring" to "old hand." And often, much too often, we know that after our students conscientiously learn what we have to offer, they walk into an organization that is not as architecturally savvy as the students now are, and our students have no practical way to put what they have learned into practice.

Most of you will not be able to dictate an architecture-based philosophy to the projects to which you'll be assigned, if it's not already present. You won't be able to say, "This Agile project needs a lead software architect!" and make it stick if the team leaders think that Agile methods won't permit any overarching design. You won't be able to say, "We're going to include an explicit architecture evaluation in our project schedule" and have everyone comply. You won't be able to say, "We're going to use the Views and Beyond approach and templates for our architecture documentation" and have your directive obeyed.

Lest you feel that all your time absorbing the material in this book was time spent for a lost cause, we want to close the book with some advice for carrying what you've learned into your professional setting:

1. **Speak the right language.** You know by now that architecture is the means to an end, and not an end in itself. The decision makers in your organization typically care about the ends, not the means. They care about products, not the architectures of those products. They care about ensuring that the products are competitive in the marketplace. And they care about executing the organization's marketing and business strategy. They may not express their concerns in architecturally significant terms, but rather in market terms that you'll have to translate.
2. **Speak the right language, part 2.** Project managers care about reduction of technical risk, reliable and realistic scheduling and budgeting, and planning the production of those products. To the extent that you can justify a focus on architecture in those terms, you'll more likely be successful in gaining the freedom to carry out some of the practices espoused in this book. And you really can justify this focus: A sound architecture is an unparalleled risk reduction strategy, a reliable work estimator, and a good predictor of production methods.
3. **Get involved.** One of the best ways to insert architectural concerns into a project is to show its value to stakeholders who don't often get a chance to see it. Requirements engineers are a case in point. Most often, they meet with customers and users, capture their concerns, write them up, and toss the requirements over the fence (usually a very tall fence) to the designers. Challenge this segregation! Try to get involved in the requirements capture activity. Invite yourself to meetings. Say that, as an architect, you want to understand the problem by hearing the concerns straight from the horse's mouth. This will give you critical exposure to the very stakeholders you'll need to help with your design, evaluation, and documentation chores. Furthermore it will give you a chance to add value to the requirements capture

process. Because you may have a design approach in mind, you may be able to offer better quality attribute responses than the customer has in mind, and that might make our marketers very happy. Or you may be able to spot troublesome requirements early on, and help nudge the customer to a perfectly adequate but more architecturally palatable QA response. Also, you can take it upon yourself to contact your organization's marketers. They are often the ones who come up with product concepts. You would do well to learn how they do that, and eventually you could help them by pointing out useful variations on existing products that could use the same architecture.

4. **It's the economy, stupid.** Think in, and couch your arguments in, economic terms. If you think an architectural trade study, or an architecture document, or an architecture evaluation, or ensuring code compliance with the architecture is a good idea for your project, make a business case for it. Pointy-haired bosses in comic strips notwithstanding, managers are really—trust us here—rational people. But their goals are broad and almost always have to do with economics. You should be able to argue, using back-of-the-envelope arithmetic, that (for example) producing an updated version of the architecture document is a worthwhile activity when the system undergoes a major change. You should be able to argue that activities undertaken with the new architecture documentation will be much less error-prone (and therefore less expensive) than those same activities undertaken without a guiding architecture. And the effort to keep the documentation up to date is much less than the expected savings. You can plug some numbers in a spreadsheet to make this argument. The numbers don't have to be accurate, just reasonable, and they'll still make your case.
5. **Start a guerrilla movement.** Find like-minded people in your organization and nurture their interest in architecture. Start a brown-bag lunchtime reading and discussion group that covers books or book chapters or papers or even blogs about architecture. For example, your group could read the chapter in this book about architecture competence, and discuss what practices you'd like to see your organization adopt, and what it would take to adopt them. Or the group could agree on a joint documentation template for architecture, or come up with a set of quality attribute scenarios that apply across your collective projects. Especially appealing is to come up with a set of patterns that apply to the systems you're building. Or bring a vexing design problem from your individual project, and let the group work on a written solution to it. Or have the group offer its services as a roving architecture evaluation team to other projects. Your group should meet regularly and often, and adopt a specific set of tangible goals. The importance of an enthusiastic and dedicated leader—you?—who is keen to mature the organization's architecture practices cannot be overstated. Advertise your

meetings, advertise your results, and keep asking more members to join your group.

6. **Relish small victories.** You don't have to change the world overnight. Every improvement you make will put you and your organization in a better place than it would have been otherwise.

Getting Architecture Reviews into an Organization through the Back Door

If you search the web for "code review computer science," you'll turn up millions of hits that describe code reviews and the steps that are taken to perform them. If you search for "design review computer science," you'll turn up little that is useful.

Other disciplines routinely practice and teach design critiques. Search for "design critique" and you will find many hits together with instructions. A design is a set of decisions of whatever type that attempts to solve a particular problem, whether an art problem, a user interface design problem, or a software problem. Solutions to important design problems should be subject to peer review, just as code should be subject to peer review.

There is a wealth of data that points out that the earlier in the life cycle a problem is discovered and fixed, the less the cost of finding and fixing the problem. Design precedes code and so having appropriate design reviews seems both intuitively and empirically justified. In addition, the documents around the review, both the original design document and also the critiques, are valuable learning tools for new developers. In many organizations developers switch systems frequently, and so they are constantly learning.

This view is not universally shared. A software engineer working in a major software house tells me that even though the organization aspires to writing and reviewing design documents, it rarely happens. Senior developers tend to limit their review to a cursory glance. Code reviews, on the other hand, are taken quite seriously by the senior developers.

My software engineer friend offers two possible explanations for this state of affairs:

1. The code review is the last opportunity to affect what is built: "review this or live with it." This explanation assumes that senior developers do not believe that the output of design reviews are actionable and thus wait to engage until later in the process.
2. The code is more concrete than the design, and is therefore easier to assess. This explanation assumes that senior developers are incapable of understanding designs.

I do not find either of these explanations compelling, but I am unable to come up with a better one.

What to do?

What this software engineer did is to look for a surrogate process where a design review could be surreptitiously performed. This individual noticed that when the organization did code reviews, questions such as "Why did you do that?" were frequently asked. The result of such questions was a discussion of rationale. So the individual would code up a solution to a problem, submit it to a code review, and wait for the question that would lead to the rationale discussion.

A design review is a review where design decisions are presented together with their rationale. Frequently, design alternatives are explored. Whether this is done under the name of code review or design review is not nearly as important as getting it done.

Of course, my friend's surreptitious approach has drawbacks. It is inefficient to code a solution that may have to be thrown away. Also, embedding design reviews into code reviews means that the designs and reviews end up being embedded in the code review tool, making it difficult to search this tool for design and design rationale. But these inefficiencies are dwarfed by the inefficiency of pursuing an incorrect solution to a particular problem.

—LB

The practice and discipline of architecture for software systems has come of age. You can be proud of joining a profession that has always strived, and is still striving, to be more disciplined, more reliable, more productive, and more efficient, to produce systems that improve the lives of their stakeholders.

With that thought, *now* it's time for the cheery book-ending imperative: Go forth and architect great systems. Your predecessors have designed systems that have changed the world. It's your turn.

This page intentionally left blank

References

[Abrahamsson 10] P. Abrahamsson, M.A. Babar, and P. Kruchten. "Agility and Architecture: Can They Coexist?" *IEEE Software*, Vol. 27, No. 2, (March-April 2010), pp. 16-22.

[AdvBuilder 10] Java Adventure Builder Reference Application. https://java.net/projects/adventurebuilder/pages/home

[Anastasopoulos 00] M. Anastasopoulos and C. Gacek. "Implementing Product Line Variabilities" (IESE-Report No. 089.00/E, V1.0). Kaiserslautern, Germany: Fraunhofer Institut Experimentelles Software Engineering, 2000.

[Anderson 08] Ross Anderson. *Security Engineering: A Guide to Building Dependable Distributed Systems, Second Edition*. Wiley, 2008.

[Argote 07]. L. Argote and G. Todorova. *International Review of Industrial and Organizational Psychology.* John Wiley & Sons, Ltd., 2007.

[Avižienis 04] Algirdas Avižienis, Jean-Claude Laprie, Brian Randell, and Carl Landwehr. "Basic Concepts and Taxonomy of Dependable and Secure Computing," *IEEE Transactions on Dependable and Secure Computing*, Vol. 1, No. 1 (January 2004), pp. 11-33.

[Bachmann 05] F. Bachmann and P. Clements. "Variability in Software Product Lines," CMU/SEI-2005-TR-012, 2005.

[Bachmann 07] Felix Bachmann, Len Bass, and Robert Nord. "Modifiability Tactics," CMU/SEI-2007-TR-002, September 2007.

[Bachmann 11] F. Bachmann. "Give the Stakeholders What They Want: Design Peer Reviews the ATAM Style," *Crosstalk*, November/December 2011, pp. 8-10, http://www.crosstalkonline.org/storage/issue-archives/2011/201111/201111-Bachmann.pdf

[Barbacci 03] M. Barbacci, R. Ellison, A. Lattanze, J. Stafford, C. Weinstock, and W. Wood. "Quality Attribute Workshops (QAWs), Third Edition," CMU/SEI-2003-TR-016, http://www.sei.cmu.edu/reports/03tr016.pdf

[Bass 03] L. Bass and B.E. John. "Linking Usability to Software Architecture Patterns through General Scenarios," *Journal of Systems and Software* 66(3), pp. 187-197.

[Bass 08] Len Bass, Paul Clements, Rick Kazman, and Mark Klein. "Models for Evaluating and Improving Architecture Competence," CMU/SEI-2008-TR-006, March 2008, http://www.sei.cmu.edu/library/abstracts/reports/08tr006.cfm

[Baudry 03] B. Baudry, Yves Le Traon, Gerson Sunyé, and Jean-Marc Jézéquel. "Measuring and Improving Design Patterns Testability," *Proceedings of the Ninth International Software Metrics Symposium* (METRICS '03), 2003.

[Baudry 05] B. Baudry and Y. Le Traon. "Measuring Design Testability of a UML Class Diagram," *Information & Software Technology* 47(13)(October 2005), pp. 859-879.

[Beck 04] Kent Beck and Cynthia Andres. *Extreme Programming Explained: Embrace Change, Second Edition*. Addison-Wesley, 2004.

[Beizer 90] B. Beizer. *Software Testing Techniques, Second Edition.* International Thomson Computer Press, 1990.

[Bellcore 98] Bell Communications Research. GR-1230-CORE, SONET Bidirectional Line-Switched Ring Equipment Generic Criteria. 1998.

[Bellcore 99] Bell Communications Research. GR-1400-CORE, SONET Dual-Fed Unidirectional Path Switched Ring (UPSR) Equipment Generic Criteria. 1999.

[Benkler 07] Y. Benkler. *The Wealth of Networks: How Social Production Transforms Markets and Freedom*. Yale University Press, 2007.

[Bertolino 96a] Antonia Bertolino and Lorenzo Strigini. "On the Use of Testability Measures for Dependability Assessment," *IEEE Transactions on Software Engineering*, Vol. 22, No. 2 (February 1996), pp. 97-108.

[Bertolino 96b] A. Bertolino and P. Inverardi. "Architecture-Based Software Testing," in *Proceedings of the Second International Software Architecture Workshop (ISAW-2)*, L. Vidal, A. Finkelstain, G. Spanoudakis, and A.L. Wolf, eds. Joint Proceedings of the SIGSOFT '96 Workshops, San Francisco, October 1996, ACM Press.

[Biffl 10] S. Biffl, A. Aurum, B. Boehm, H. Erdogmus, and P. Grunbacher, eds. *Value-Based Software Engineering*. Springer, 2010.

[Binder 94] R.V. Binder. "Design for Testability in Object-Oriented Systems," *CACM* 37(9), pp. 87-101, 1994.

[Boehm 78] B.W. Boehm, J.R. Brown, J.R. Kaspar, M.L. Lipow, and G. MacCleod. *Characteristics of Software Quality*. American Elsevier, 1978.

[Boehm 81] B. Boehm. *Software Engineering Economics*. Prentice-Hall, 1981.

[Boehm 91] Barry Boehm. "Software Risk Management: Principles and Practices," *IEEE Software*, Vol. 8, No. 1, pp. 32-41, January 1991.

[Boehm 04] B. Boehm and R. Turner. *Balancing Agility and Discipline: A Guide for the Perplexed*. Addison-Wesley, 2004.

[Boehm 07] B. Boehm, R. Valerdi, and E. Honour. "The ROI of Systems Engineering: Some Quantitative Results for Software Intensive Systems," *Systems Engineering*, Vol. 11, No. 3, pp. 221-234.

[Boehm 10] B. Boehm, J. Lane, S. Koolmanojwong, and R. Turner. "Architected Agile Solutions for Software-Reliant Systems," Technical Report USC-CSSE-2010-516, 2010.

[Booch 11] Grady Booch. "An Architectural Oxymoron," podcast available at http://www.computer.org/portal/web/computingnow/onarchitecture. Retrieved January 21, 2011.

[Bosch 00] J. Bosch. "Organizing for Software Product Lines," *Proceedings of the 3rd International Workshop on Software Architectures for Product Families (IWSAPF-3)*, pp. 117-134. Las Palmas de Gran Canaria, Spain, March 15-17, 2000. Springer, 2000.

[Bouwers 10] E. Bouwers and A. van Deursen. "A Lightweight Sanity Check for Implemented Architectures," *IEEE Software* 27(4), July/August 2010, pp. 44-50.

[Bredemeyer 11] D. Bredemeyer and R. Malan. "Architect Competencies: What You Know, What You Do and What You Are," http://www.bredemeyer.com/Architect/ArchitectSkillsLinks.htm

[Brewer 12] E. Brewer. "CAP Twelve Years Later: How the 'Rules' Have Changed," *IEEE Computer*, February 2012, pp. 23-29.

[Brown 10] N. Brown, R. Nord, and I. Ozkaya. "Enabling Agility Through Architecture," *Crosstalk*, November/December 2010, pp. 12-17.

[Brownsword 96] Lisa Brownsword and Paul Clements. "A Case Study in Successful Product Line Development," Technical Report CMU/SEI-96-TR-016, October 1996.

[Brownsword 04] Lisa Brownsword, David Carney, David Fisher, Grace Lewis, Craig Meterys, Edwin Morris, Patrick Place, James Smith, and Lutz Wrage. "Current Perspectives on Interoperability," CMU/SEI-2004-TR-009, http://www.sei.cmu.edu/reports/04tr009.pdf

[Bruntink 06] Magiel Bruntink and Arie van Deursen. "An Empirical Study into Class Testability," *Journal of Systems and Software* 79(9)(2006), pp. 1219-1232.

[Buschmann 96] Frank Buschmann, Regine Meunier, Hans Rohnert, Peter Sommerlad, and Michael Stal. *Pattern-Oriented Software Architecture Volume 1: A System of Patterns.* Wiley, 1996.

[Cai 11] Yuanfang Cai, Daniel Iannuzzi, and Sunny Wong. "Leveraging Design Structure Matrices in Software Design Education," *Conference on Software Engineering Education and Training 2011*, pp. 179-188.

[Cappelli 12] Dawn M. Cappelli, Andrew P. Moore, and Randall F. Trzeciak. *The CERT Guide to Insider Threats: How to Prevent, Detect, and Respond to Information Technology Crimes (Theft, Sabotage, Fraud)*. Addison-Wesley, 2012.

[Carriere 10] J. Carriere, R. Kazman, and I. Ozkaya. "A Cost-Benefit Framework for Making Architectural Decisions in a Business Context," *Proceedings of 32nd International Conference on Software Engineering (ICSE 32)*, Capetown, South Africa, May 2010.

[Cataldo 07] M. Cataldo, M. Bass, J. Herbsleb, and L. Bass. "On Coordination Mechanisms in Global Software Development," *Proceedings Second IEEE International Conference on Global Software Development*, 2007.

[Chandran 10] S. Chandran, A. Dimov, and S. Punnekkat. "Modeling Uncertainties in the Estimation of Software Reliability—A Pragmatic Approach," *Fourth IEEE International Conference on Secure Software Integration and Reliability Improvement*, 2010.

[Chang 06] F. Chang, J. Dean, S. Ghemawat, W. Hsieh, et al. "Bigtable: A Distributed Storage System for Structured Data," *Proceedings Operating Systems Design and Implementation*, 2006, http://research.google.com/archive/bigtable.html

[Chen 10] H.-M. Chen, R. Kazman, and O. Perry. "From Software Architecture Analysis to Service Engineering: An Empirical Study of Enterprise SOA Implementation," *IEEE Transactions on Services Computing* 3(2)(April-June 2010), pp. 145-160.

[Chidamber 94] S. Chidamber and C. Kemerer. "A Metrics Suite for Object Oriented Design," *IEEE Transactions on Software Engineering*, Vol. 20, No. 6 (June 1994).

[Clements 01a] P. Clements and L. Northrop. *Software Product Lines.* Addison-Wesley, 2001.

[Clements 01b] P. Clements, R. Kazman, and M. Klein. *Evaluating Software Architectures.* Addison-Wesley, 2001.

[Clements 07] P. Clements, R. Kazman, M. Klein, D. Devesh, S. Reddy, and P. Verma. "The Duties, Skills, and Knowledge of Software Architects," *Proceedings of the Working IEEE/IFIP Conference on Software Architecture*, 2007.

[Clements 10a] Paul Clements, Felix Bachmann, Len Bass, David Garlan, James Ivers, Reed Little, Paulo Merson, Robert Nord, and Judith Stafford. *Documenting Software Architectures: Views and Beyond, Second Edition.* Addison-Wesley, 2010.

[Clements 10b] Paul Clements and Len Bass. "Relating Business Goals to Architecturally Significant Requirements for Software Systems," CMU/SEI-2010-TN-018, May 2010.

[Clements 10c] P. Clements and L. Bass. "The Business Goals Viewpoint," *IEEE Software* 27(6)(November-December 2010), pp. 38-45.

[Cockburn 04] Alistair Cockburn. *Crystal Clear: A Human-Powered Methodology for Small Teams.* Addison-Wesley, 2004.

[Conway 68] Melvin E. Conway. "How Do Committees Invent?" *Datamation*, Vol. 14, No. 4 (1968), pp. 28-31.

[Coplein 10] J. Coplein and G. Bjornvig. *Lean Architecture for Agile Software Development.* Wiley, 2010.

[Cunningham 92] W. Cunningham. "The Wycash Portfolio Management System," in Addendum to the *Proceedings of Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, pp. 29-30, ACM Press, 1992.

[CWE 12] The Common Weakness Enumeration. http://cwe.mitre.org/

[Dean 04] Jeffrey Dean and Sanjay Ghemawat. "MapReduce: Simplified Data Processing on Large Clusters," *Proceedings Operating System Design and Implementation*, 1994, http://research.google.com/archive/mapreduce.html

[Dijkstra 68] E.W. Dijkstra. "The Structure of the 'THE'-Multiprogramming System," *Communications of the ACM* 11(5), pp. 341-346.

[Dix 04] Alan Dix, Janet Finlay, Gregory Abowd, and Russell Beale. *Human-Computer Interaction, Third Edition*. Prentice Hall, 2004.

[Douglass 99] Bruce Douglass. *Real-Time Design Patterns: Robust Scalable Architecture for Real-Time Systems*. Addison-Wesley, 1999.

[Dutton 84] J.M. Dutton and A. Thomas. "Treating Progress Functions as a Managerial Opportunity," *Academy of Management Review* 9 (1984), pp. 235-247.

[Eickelman 96] N. Eickelman and D. Richardson. "What Makes One Software Architecture More Testable Than Another?" in *Proceedings of the Second International Software Architecture Workshop (ISAW-2)*, L. Vidal, A. Finkelstein, G. Spanoudakis, and A.L. Wolf, eds., Joint Proceedings of the SIGSOFT '96 Workshops, San Francisco, October 1996, ACM Press.

[EOSAN 07] "WP 8.1.4—Define Methodology for Validation within OATA: Architecture Tactics Assessment Process," http://www.eurocontrol.int/valfor/gallery/content/public/OATA-P2-D8.1.4-01%20DMVO%20Architecture%20Tactics%20Assessment%20Process.pdf

[FAA 00] "System Safety Handbook," http://www.faa.gov/library/manuals/aviation/risk_management/ss_handbook/

[Fairbanks 10] G. Fairbanks. *Just Enough Software Architecture*. Marshall & Brainerd, 2010.

[Feiler 06] P. Feiler, R.P. Gabriel, J. Goodenough, R. Linger, T. Longstaff, R. Kazman, M. Klein, L. Northrop, D. Schmidt, K. Sullivan, and K. Wallnau. *Ultra-Large-Scale Systems: The Software Challenge of the Future*, http://www.sei.cmu.edu/library/assets/ULS_Book20062.pdf

[Fiol 85] C.M. Fiol and M.A. Lyles. "Organizational Learning," *Academy of Management Review* 10(4)(1985), p. 803.

[Freeman 09] Steve Freeman and Nat Pryce. *Growing Object-Oriented Software, Guided by Tests*. Addison-Wesley, 2009.

[Gacek 95] Cristina Gacek, Ahmed Abd-Allah, Bradford Clark, and Barry Boehm. "On the Definition of Software System Architecture," USC/CSE-95-TR-500, April 1995.

[Gagliardi 09] M. Gagliardi, W. Wood, J. Klein, and J. Morley. "A Uniform Approach for System of Systems Architecture Evaluation," *Crosstalk*, Vol. 22, No. 3 (March/April 2009), pp. 12-15.

[Gamma 94] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1994.

[Garlan 93] D. Garlan and M. Shaw. "An Introduction to Software Architecture," in Ambriola and Tortola, eds., *Advances in Software Engineering & Knowledge Engineering, Vol. II*. World Scientific Pub. Co., 1993, pp. 1-39.

[Garlan 95] David Garlan, Robert Allen, and John Ockerbloom. "Architectural Mismatch or Why it's hard to build systems out of existing parts," ICSE 1995. 17th International Conference on Software Engineering, April 1995.

[Gilbert 07] T. Gilbert. *Human Competence: Engineering Worthy Performance.* Pfeiffer, Tribute Edition, 2007.

[Gokhale 05] S. Gokhale, J. Crigler, W. Farr, and D. Wallace. "System Availability Analysis Considering Hardware/Software Failure Severities," *Proceedings of the 29th Annual IEEE/NASA Software Engineering Workshop (SEW '05)*, Greenbelt, MD, April 2005, IEEE 2005.

[Gorton 10] Ian Gorton. *Essential Software Architecture, Second Edition.* Springer, 2010.

[Graham 07] T.C.N. Graham, R. Kazman, and C. Walmsley. "Agility and Experimentation: Practical Techniques for Resolving Architectural Tradeoffs," *Proceedings of the 29th International Conference on Software Engineering (ICSE 29)*, Minneapolis, MN, May 2007.

[Gray 93] Jim Gray and Andreas Reuter. *Distributed Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1993.

[Grinter 99] Rebecca E. Grinter. "Systems Architecture: Product Designing and Social Engineering," in *Proceedings of the International Joint Conference on Work Activities Coordination and Collaboration (WACC '99)*, Dimitrios Georgakopoulos, Wolfgang Prinz, and Alexander L. Wolf, eds. ACM, 1999, pp. 11-18.

[Hamm 04] "Linus Torvalds' Benevolent Dictatorship," *BusinessWeek*, August 18, 2004, http://www.businessweek.com/technology/content/aug2004/tc20040818_1593.htm

[Hamming 80] R.W. Hamming. *Coding and Information Theory*. Prentice Hall, 1980.

[Hanmer 07] Robert Hanmer. *Patterns for Fault Tolerant Software*, Wiley, 2007.

[Harms 10] R. Harms and M. Yamartino. "The Economics of the Cloud," http://economics.uchicago.edu/pdf/Harms_110111.pdf

[Hartman 10] Gregory Hartman. "Attentiveness: Reactivity at Scale," CMU-ISR-10-111, 2010.

[Hiltzik 00] M. Hiltzik. *Dealers of Lightning: Xerox PARC and the Dawn of the Computer Age*. Harper Business, 2000.

[Hoffman 00] Daniel M. Hoffman and David M. Weiss. *Software Fundamentals: Collected Papers by David L. Parnas*. Addison-Wesley, 2000.

[Hofmeister 00] Christine Hofmeister, Robert Nord, and Dilip Soni. *Applied Software Architecture*. Addison-Wesley, 2000.

[Hofmeister 07] Christine Hofmeister, Philippe Kruchten, Robert L. Nord, Henk Obbink, Alexander Ran, and Pierre America. "A General Model of Software Architecture Design Derived from Five Industrial Approaches," *Journal of Systems and Software*, Vol. 80, No. 1 (January 2007), pp. 106-126.

[Howard 04] Michael Howard. "Mitigate Security Risks by Minimizing the Code You Expose to Untrusted Users," *MSDN Magazine*, http://msdn.microsoft.com/en-us/magazine/cc163882.aspx

[IEEE 94] "IEEE Standard for Software Safety Plans," STD-1228-1994, http://standards.ieee.org/findstds/standard/1228-1994.html

[IEEE 11] "IEEE Guide—Adoption of the Project Management Institute (PMI) Standard: A Guide to the Project Management Body of Knowledge (PMBOK Guide), Fourth Edition," http://www.projectsmart.co.uk/pmbok.html

[IETF 04] Internet Engineering Task Force. "RFC 3746, Forwarding and Control Element Separation (ForCES) Framework," 2004.

[IETF 05] Internet Engineering Task Force. "RFC 4090, Fast Reroute Extensions to RSVP-TE for LSP Tunnels," 2005.

[IETF 06a] Internet Engineering Task Force. "RFC 4443, Internet Control Message Protocol (ICMPv6) for the Internet Protocol Version 6 (IPv6) Specification," 2006.

[IETF 06b] Internet Engineering Task Force. "RFC 4379, Detecting Multi-Protocol Label Switched (MPLS) Data Plane Failures," 2006.

[INCOSE 05] International Council on Systems Engineering. "System Engineering Competency Framework 2010-0205," http://www.incose.org/ProductsPubs/products/competenciesframework.aspx

[ISO 11] International Organization for Standardization. "ISO/IEC 25010: 2011 Systems and software engineering—Systems and software Quality Requirements and Evaluation (SQuaRE)—System and software quality models."

[Jacobson 97] I. Jacobson, M. Griss, and P. Jonsson. *Software Reuse: Architecture, Process, and Organization for Business Success*. Addison-Wesley, 1997.

[Kanwal 10] F. Kanwal, K. Junaid, and M.A. Fahiem. "A Hybrid Software Architecture Evaluation Method for FDD—An Agile Process Mode," *2010 International Conference on Computational Intelligence and Software Engineering (CiSE)*, December 2010, pp. 1-5.

[Kaplan 92] R. Kaplan and D. Norton. "The Balanced Scorecard: Measures That Drive Performance," *Harvard Business Review*, January/February 1992, pp. 71-79.

[Karat 94] Claire Marie Karat. "A Business Case Approach to Usability Cost Justification," in *Cost-Justifying Usability*, R. Bias and D. Mayhew, eds. Academic Press, 1994.

[Kazman 94] Rick Kazman, Len Bass, Mike Webb, and Gregory Abowd. "SAAM: A Method for Analyzing the Properties of Software Architectures," in *Proceedings of the 16th International Conference on Software Engineering (ICSE '94)*. Los Alamitos, CA. IEEE Computer Society Press, pp. 81-90.

[Kazman 99] R. Kazman and S.J. Carriere. "Playing Detective: Reconstructing Software Architecture from Available Evidence," *Automated Software Engineering* 6(2)(April 1999), pp. 107-138.

[Kazman 01] R. Kazman, J. Asundi, and M. Klein. "Quantifying the Costs and Benefits of Architectural Decisions," *Proceedings of the 23rd International Conference on Software Engineering (ICSE 23)*, Toronto, Canada, May 2001, pp. 297-306.

[Kazman 02] R. Kazman, L. O'Brien, and C. Verhoef. "Architecture Reconstruction Guidelines, Third Edition," CMU/SEI Technical Report, CMU/SEI-2002-TR-034, 2002.

[Kazman 04] R. Kazman, P. Kruchten, R. Nord, and J. Tomayko. "Integrating Software-Architecture-Centric Methods into the Rational Unified Process," Technical Report CMU/SEI-2004-TR-011, July 2004, http://www.sei.cmu.edu/library/abstracts/reports/04tr011.cfm

[Kazman 05] Rick Kazman and Len Bass. "Categorizing Business Goals for Software Architectures," CMU/SEI-2005-TR-021, December 2005.

[Kazman 09] R. Kazman and H.-M. Chen. "The Metropolis Model: A New Logic for the Development of Crowdsourced Systems," *Communications of the ACM*, July 2009, pp. 76-84.

[Kircher 03] Michael Kircher and Prashant Jain. *Pattern-Oriented Software Architecture Volume 3: Patterns for Resource Management.* Wiley, 2003.

[Klein 10] J. Klein and M. Gagliardi. "A Workshop on Analysis and Evaluation of Enterprise Architectures," CMU/SEI-2010-TN-023, http://www.sei.cmu.edu/reports/10tn023.pdf

[Klein 93] M. Klein, T. Ralya, B. Pollak, R. Obenza, and M. Gonzalez Harbour. *A Practitioner's Handbook for Real-Time Systems Analysis*. Kluwer Academic, 1993.

[Koziolet 10] H. Koziolek. "Performance Evaluation of Component-Based Software Systems: A Survey," *Performance Evaluation* 67(8)(August 2010).

[Kruchten 95] P.B. Kruchten. "The 4+1 View Model of Architecture," *IEEE Software*, Vol. 12, No. 6 (November 1995), pp. 42-50.

[Kruchten 03] Philippe Kruchten. *The Rational Unified Process: An Introduction, Third Edition.* Addison-Wesley, 2003.

[Kruchten 04] Philippe Kruchten. "An Ontology of Architectural Design Decisions," in Jan Bosch, ed., *Proceedings of the 2nd Workshop on Software Variability Management*, Groningen, NL, Dec. 3-4, 2004.

[Kumar 10a] K. Kumar and TV Prabhakar. "Pattern-Oriented Knowledge Model for Architecture Design," in *Pattern Languages of Programs Conference 2010*, October 15-18, 2010, Reno/Tahoe, Nevada.

[Kumar 10b] Kiran Kumar and TV Prabhakar. "Design Decision Topology Model for Pattern Relationship Analysis," *Asian Conference on Pattern Languages of Programs 2010*, March 15-17, 2010, Tokyo, Japan.

[Ladas 09] Corey Ladas. *Scrumban: Essays on Kanban Systems for Lean Software Development*. Modus Cooperandi Press, 2009.

[Lattanze 08] Tony Lattanze. *Architecting Software Intensive Systems: A Practitioner's Guide*. Auerbach Publications, 2008.

[Le Traon 97] Y. Le Traon and C. Robach. "Testability Measurements for Data Flow Designs," *Proceedings of the 4th International Symposium on Software Metrics (METRICS '97)*, pp. 91-98. November 1997, Washington, D.C.

[Leveson 04] Nancy G. Leveson. "The Role of Software in Spacecraft Accidents," *Journal of Spacecraft and Rockets* 41(4)(July 2004), pp. 564-575.

[Leveson 11] Nancy G. Leveson. *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.

[Levitt 88] B. Levitt and J. March. "Organizational Learning," *Annual Review of Sociology* 14 (1988), pp. 319-340.

[Liu 00] Jane Liu. *Real-Time Systems.* Prentice Hall, 2000.

[Liu 09] Henry Liu. *Software Performance and Scalability: A Quantitative Approach*. Wiley, 2009.

[Luftman 00] J. Luftman. "Assessing Business Alignment Maturity," *Communications of AIS*, Vol. 4, No. 14, 2000.

[Lyons 62] R. E. Lyons and W. Vanderkulk. "The Use of Triple-Modular Redundancy to Improve Computer Reliability," *IBM J. Res. Dev.* 6(2)(April 1962), pp. 200-209.

[MacCormack 06] A. MacCormack, J. Rusnak, and C. Baldwin. "Exploring the Structure of Complex Software Designs: An Empirical Study of Open Source and Proprietary Code," *Management Science* 52(7)(July 2006), pp. 1015-1030.

[MacCormack 10] A. MacCormack, C. Baldwin, and J. Rusnak. "The Architecture of Complex Systems: Do Core-Periphery Structures Dominate?" MIT Sloan Research Paper No. 4770-10, http://www.hbs.edu/research/pdf/10-059.pdf

[Malan 00] Ruth Malan and Dana Bredemeyer. "Creating an Architectural Vision: Collecting Input," http://www.bredemeyer.com/pdf_files/vision_input.pdf, July 25, 2000.

[Maranzano 05] Joseph F. Maranzano, Sandra A. Rozsypal, Gus H. Zimmerman, Guy W. Warnken, Patricia E. Wirth, and David M. Weiss. "Architecture Reviews: Practice and Experience," *IEEE Software*, March/April 2005, pp. 34-43.

[Mavis 02] D.G. Mavis. "Soft Error Rate Mitigation Techniques for Modern Microcircuits," *40th Annual Reliability Physics Symposium Proceedings*, April 2002, Dallas, TX. IEEE, 2002.

[McCall 77] J.A. McCall, P.K. Richards, and G.F. Walters. *Factors in Software Quality*. Griffiths Air Force Base, N.Y. : Rome Air Development Center Air Force Systems Command.

[McGregor 11] John D. McGregor, J. Yates Monteith, and Jie Zhang. "Quantifying Value in Software Product Line Design," in *Proceedings of the 15th International Software Product Line Conference, Volume 2 (SPLC '11)*, Ina Schaefer, Isabel John, and Klaus Schmid, eds.

[Mettler 91] R. Mettler. "Frederick C. Lindvall," in *Memorial Tributes: National Academy of Engineering, Volume 4*, pp. 213-216. National Academy of Engineering, 1991.

[Moore 03] M. Moore, R. Kazman, M. Klein, and J. Asundi. "Quantifying the Value of Architecture Design Decisions: Lessons from the Field," *Proceedings of the 25th International Conference on Software Engineering (ICSE 25)*, Portland, OR, May 2003, pp. 557-562.

[Morelos-Zaragoza 06] R.H. Morelos-Zaragoza. *The Art of Error Correcting Coding, Second Edition*. Wiley, 2006.

[Muccini 03] H. Muccini, A. Bertolino, and P. Inverardi. "Using Software Architecture for Code Testing," *IEEE Transactions on Software Engineering* 30(3), pp. 160-171.

[Muccini 07] H. Muccini. "What Makes Software Architecture-Based Testing Distinguishable," in *Proc. Sixth Working IEEE/IFIP Conference on Software Architecture*, WICSA 2007, Mumbai, India, January 2007.

[Murphy 01] G. Murphy, D. Notkin, and K. Sullivan. "Software Reflexion Models: Bridging the Gap between Design and Implementation," *IEEE Transactions on Software Engineering*, Vol. 27, pp. 364-380, 2001.

[Nielsen 08] Jakob Nielsen. "Usability ROI Declining, But Still Strong," http://www.useit.com/alertbox/roi.html

[NIST 02] National Institute of Standards and Technology. "Security Requirements For Cryptographic Modules," FIPS Pub. 140-2, http://csrc.nist.gov/publications/fips/fips140-2/fips1402.pdf

[NIST 04] National Institute of Standards and Technology. "Standards for Security Categorization of Federal Information Systems," FIPS Pub. 199, http://csrc.nist.gov/publications/fips/fips199/FIPS-PUB-199-final.pdf

[NIST 06] National Institute of Standards and Technology. "Minimum Security Requirements for Federal Information and Information Systems," FIPS Pub. 200, http://csrc.nist.gov/publications/fips/fips200/FIPS-200-final-march.pdf

[NIST 09] National Institute of Standards and Technology. "800-53 v3 Recommended Security Controls for Federal Information Systems and Organizations," August 2009, http://csrc.nist.gov/publications/nistpubs/800-53-Rev3/sp800-53-rev3-final.pdf

[Nord 04] R. Nord, J. Tomayko, and R. Wojcik. "Integrating Software Architecture-Centric Methods into Extreme Programming (XP)," CMU/SEI-2004-TN-036. Software Engineering Institute, Carnegie Mellon University, 2004.

[Nygard 07] Michael T. Nygard. *Release It!: Design and Deploy Production-Ready Software*. Pragmatic Programmers, 2007.

[Obbink 02] H. Obbink, P. Kruchten, W. Kozaczynski, H. Postema, A. Ran, L. Dominic, R. Kazman, R. Hilliard, W. Tracz, and E. Kahane. "Software Architecture Review and Assessment (SARA) Report, Version 1.0," 2002, http://pkruchten.wordpress.com/architecture/SARAv1.pdf/

[O'Brien 03] L. O'Brien and C. Stoermer. "Architecture Reconstruction Case Study," CMU/SEI Technical Note, CMU/SEI-2003-TN-008, 2003.

[ODUSD 08] Office of the Deputy Under Secretary of Defense for Acquisition and Technology. "Systems Engineering Guide for Systems of Systems, Version 1.0," 2008, http://www.acq.osd.mil/se/docs/SE-Guide-for-SoS.pdf

[Palmer 02] Stephen Palmer and John Felsing. *A Practical Guide to Feature-Driven Development.* Prentice Hall, 2002.

[Parnas 72] D.L. Parnas. "On the Criteria to Be Used in Decomposing Systems into Modules," *Communications of the ACM* 15(12)(December 1972).

[Parnas 74] D. Parnas. "On a 'Buzzword': Hierarchical Structure," *Proceedings IFIP Congress 74*, pp. 336-339. North Holland Publishing Company, 1974.

[Parnas 76] D.L. Parnas. "On the Design and Development of Program Families," *IEEE Transactions on Software Engineering*, SE-2, 1 (March 1976), pp. 1-9.

[Parnas 79] D. Parnas. "Designing Software for Ease of Extension and Contraction," *IEEE Transactions on Software Engineering,* SE-5, 2 (1979), pp. 128-137.

[Parnas 95] David Parnas and Jan Madey. "Functional Documents for Computer Systems," chapter in *Science of Computer Programming.* Elsevier, 1995.

[Paulish 02] Daniel J. Paulish. *Architecture-Centric Software Project Management: A Practical Guide.* Addison-Wesley, 2002.

[Pena 87] William Pena. *Problem Seeking: An Architectural Programming Primer.* AIA Press, 1987.

[Perry 92] Dewayne E. Perry and Alexander L. Wolf. "Foundations for the Study of Software Architecture," *SIGSOFT Softw. Eng. Notes* 17(4)(October 1992), pp. 40-52.

[Pettichord 02] B. Pettichord. "Design for Testability," Pacific Northwest Software Quality Conference, Portland, Oregon, October 2002.

[Powel Douglass 99] B. Powel Douglass. *Doing Hard Time: Developing Real-Time Systems with UML, Objects, Frameworks, and Patterns.* Addison-Wesley, 1999.

[Sangwan 08] Raghvinder Sangwan, Colin Neill, Matthew Bass, and Zakaria El Houda. "Integrating a Software Architecture-Centric Method into Object-Oriented Analysis and Design," *Journal of Systems and Software*, Vol. 81, No. 5 (May 2008), pp. 727-746.

[Schmerl 06] B. Schmerl, J. Aldrich, D. Garlan, R. Kazman, and H. Yan. "Discovering Architectures from Running Systems," *IEEE Transactions on Software Engineering* 32(7)(July 2006), pp. 454-466.

[Schmidt 00] Douglas Schmidt, M. Stal, H. Rohnert, and F. Buschmann. *Pattern-Oriented Software Architecture: Patterns for Concurrent and Networked Objects*. Wiley, 2000.

[Schmidt 10] Klaus Schmidt. *High Availability and Disaster Recovery: Concepts, Design, Implementation.* Springer, 2010.

[Schneier 96] B. Schneier. *Applied Cryptography*. Wiley, 1996.

[Schneier 08] Bruce Schneier. *Schneier on Security.* Wiley, 2008.

[Schwaber 04] Ken Schwaber. *Agile Project Management with Scrum.* Microsoft Press, 2004.

[Scott 09] James Scott and Rick Kazman. "Realizing and Refining Architectural Tactics: Availability," Technical Report CMU/SEI-2009-TR-006, August 2009.

[Seacord 05] Robert Seacord. *Secure Coding in C and C++*. Addison-Wesley, 2005.

[SEI 12] Software Engineering Institute. "A Framework for Software Product Line Practice, Version 5.0," http://www.sei.cmu.edu/productlines/frame_report/PL.essential.act.htm

[Shaw 94] Mary Shaw. "Procedure Calls Are the Assembly Language of Software Interconnections: Connectors Deserve First-Class Status," Carnegie Mellon University Technical Report, 1994, http://repository.cmu.edu/cgi/viewcontent.cgi?article=1234&context=sei

[Shaw 95] Mary Shaw. "Beyond Objects: A Software Design Paradigm Based on Process Control," *ACM Software Engineering Notes*, Vol. 20, No. 1 (January 1995), pp. 27-38.

[Smith 01] Connie U. Smith and Lloyd G. Williams. *Performance Solutions: A Practical Guide to Creating Responsive, Scalable Software*. Addison-Wesley, 2001.

[Soni 95] Dilip Soni, Robert L. Nord, and Christine Hofmeister. "Software Architecture in Industrial Applications," *International Conference on Software Engineering 1995*, April 1995, pp. 196-207.

[Stonebraker 09] M. Stonebraker. "The 'NoSQL' Discussion Has Nothing to Do with SQL," http://cacm.acm.org/blogs/blog-cacm/50678-the-nosql-discussion-has-nothing-to-do-with-sql/fulltext

[Stonebraker 10a] M. Stonebraker. "SQL Databases v. NoSQL Databases," *Communications of the ACM* 53(4), p. 10.

[Stonebraker 10b] M. Stonebraker, D. Abadi, D.J. Dewitt, S. Madden, E. Paulson, A. Pavlo, and A. Rasin. "MapReduce and Parallel DBMSs," *Communications of the ACM* 53, p. 6.

[Stonebraker 11] M. Stonebraker. "Stonebraker on NoSQL and Enterprises," *Communications of the ACM* 54(8), p. 10.

[Storey 97] M.-A. Storey, K. Wong, and H. Müller. "Rigi—A Visualization Environment for Reverse Engineering (Research Demonstration Summary)," *19th International Conference on Software Engineering (ICSE 97)*, May 1997, pp. 606-607. IEEE Computer Society Press.

[Svahnberg 00] M. Svahnberg and J. Bosch. "Issues Concerning Variability in Software Product Lines," in *Proceedings of the Third International Workshop on Software Architectures for Product Families,* Las Palmas de Gran Canaria, Spain, March 15-17, 2000, pp. 50-60. Springer, 2000.

[Taylor 09] R. Taylor, N. Medvidovic, and E. Dashofy. *Software Architecture: Foundations, Theory, and Practice*. Wiley, 2009.

[Telcordia 00] Telcordia. "GR-253-CORE, Synchronous Optical Network (SONET) Transport Systems: Common Generic Criteria." 2000.

[Urdangarin 08] R. Urdangarin, P. Fernandes, A. Avritzer, and D. Paulish. "Experiences with Agile Practices in the Global Studio Project," *Proceedings of the IEEE International Conference on Global Software Engineering*, 2008.

[Utas 05] G. Utas. *Robust Communications Software: Extreme Availability, Reliability, and Scalability for Carrier-Grade Systems*. Wiley, 2005.

[van der Linden 07] F. van der Linden, K. Schmid, and E. Rommes. *Software Product Lines in Action.* Springer, 2007.

[van Deursen 04] A. van Deursen, C. Hofmeister, R. Koschke, L. Moonen, and C. Riva. "Symphony: View-Driven Software Architecture Reconstruction," *Proceedings of the 4th Working IEEE/IFIP Conference on Software Architecture (WICSA 2004)*, June 2004, Oslo, Norway. IEEE Computer Society.

[van Vliet 05] H. van Vliet. "The GRIFFIN project, A GRId For inFormatIoN about architectural knowledge," http://griffin.cs.vu.nl/, Vrije Universiteit, Amsterdam, April 16, 2005.

[Verizon 12] "Verizon 2012 Data Breach Investigations Report," http://www.verizonbusiness.com/resources/reports/rp_data-breach-investigations-report-2012_en_xg.pdf

[Vesely 81] W.E. Vesely, F.F. Goldberg, N.H. Roberts, and D.F. Haasl. "Fault Tree Handbook," http://www.nrc.gov/reading-rm/doc-collections/nuregs/staff/sr0492/sr0492.pdf

[Vesely 02] William Vesely, Michael Stamatelatos, Joanne Dugan, Joseph Fragola, Joseph Minarick III, and Jan Railsback. "Fault Tree Handbook with Aerospace Applications," http://www.hq.nasa.gov/office/codeq/doctree/fthb.pdf

[Viega 01] John Viega and Gary McGraw. *Building Secure Software: How to Avoid Security Problems the Right Way.* Addison-Wesley, 2001.

[Voas 95] Jeffrey M. Voas and Keith W. Miller. "Software Testability: the New Verification," *IEEE Software* 12(3)(May 1995), pp. 17-28.

[Von Neumann 56] J. Von Neumann. "Probabilistic Logics and the Synthesis of Reliable Organisms from Unreliable Components," *Automata Studies*, C.E. Shannon and J. McCarthy, eds. Princeton University Press, 1956.

[Wojcik 06] R. Wojcik, F. Bachmann, L. Bass, P. Clements, P. Merson, R. Nord, and W. Wood. "Attribute-Driven Design (ADD), Version 2.0," Technical Report CMU/SEI-2006-TR-023, November 2006, http://www.sei.cmu.edu/library/abstracts/reports/06tr023.cfm

[Wood 07] W. Wood. "A Practical Example of Applying Attribute-Driven Design (ADD), Version 2.0," Technical Report CMU/SEI-2007-TR-005, February 2007, http://www.sei.cmu.edu/library/abstracts/reports/07tr005.cfm

[Woods 11] E. Woods and N. Rozanski. *Software Systems Architecture: Working with Stakeholders Using Viewpoints and Perspectives, Second Edition*. Addison-Wesley, 2011.

[Wozniak 07] J. Wozniak, V. Baggiolini, D. Garcia Quintas, and J. Wenninger. "Software Interlocks System," *Proceedings ICALEPCS07*, http://ics-web4.sns.ornl.gov/icalepcs07/WPPB03/WPPB03.PDF

[Wu 06] W. Wu and T. Kelly. "Deriving Safety Requirements as Part of System Architecture Definition," in *Proceedings of 24th International System Safety Conference*, published by the System Safety Society, August 2006, Albuquerque, NM.

[Yacoub 02] S. Yacoub and H. Ammar. "A Methodology for Architecture-Level Reliability Risk Analysis," *IEEE Transactions on Software Engineering*, Vol. 28, No. 6 (June 2002).

[Yin 94] James Bieman and Hwei Yin. "Designing for Software Testability Using Automated Oracles," *Proceedings International Test Conference*, September 1992, pp. 900-907.

This page intentionally left blank

About the Authors

Len Bass is a Senior Principal Researcher at National ICT Australia Ltd. (NICTA). He joined NICTA in 2011 after 25 years at the Software Engineering Institute (SEI) at Carnegie Mellon University. He is the coauthor of two award-winning books in software architecture, including *Documenting Software Architectures: Views and Beyond, Second Edition* (Addison-Wesley, 2011), as well as several other books and numerous papers in computer science and software engineering on a wide range of topics. Len has almost 50 years' experience in software development and research in multiple domains, such as scientific analysis systems, embedded systems, and information systems.

Paul Clements is the Vice President of Customer Success at BigLever Software, Inc., where he works to spread the adoption of systems and software product line engineering. Prior to this position, he was Senior Member of the Technical Staff at the SEI, where for 17 years he was leader or co-leader of projects in software product line engineering and software architecture documentation and analysis. Other books Paul has coauthored include *Documenting Software Architectures: Views and Beyond, Second Edition* (Addison-Wesley, 2011), *Evaluating Software Architectures: Methods and Case Studies* (Addison-Wesley, 2002), and *Software Product Lines: Practices and Patterns* (Addison-Wesley, 2002). In addition, he has also published dozens of papers in software engineering reflecting his long-standing interest in the design and specification of challenging software systems. Paul was a founding member of the IFIP WG2.10 Working Group on Software Architecture.

Rick Kazman is a Professor at the University of Hawaii and a Visiting Scientist (and former Senior Member of the Technical Staff) at the SEI. He is a coauthor of *Evaluating Software Architectures: Methods and Case Studies* (Addison-Wesley, 2002) and author of more than 100 technical papers. Rick's primary research interests focus on software architecture, design and analysis, software visualization, and software engineering economics. Rick has created several highly influential methods and tools for architecture analysis, including the SAAM (Software Architecture Analysis Method), the ATAM (Architecture Tradeoff Analysis Method), the CBAM (Cost-Benefit Analysis Method), and the Dali architecture reverse-engineering tool.

This page intentionally left blank